

UNIVERSITI TUNKU ABDUL RAHMAN

ACADEMIC YEAR 2023/2024

MAY EXAMINATION

UCCD3074 DEEP LEARNING FOR DATA SCIENCE

SATURDAY, 18 MAY 2024

TIME : 2.00 PM – 4.00 PM (2 HOURS)

BACHELOR OF COMPUTER SCIENCE (HONOURS)

Instructions to Candidates:

This question paper consists of FIVE (5) questions.

Answer **ALL** questions in **Section A** and **ONLY ONE (1)** question in **Section B**. Each question carries 25 marks.

Should a candidate answers more than ONE (1) question in section B, marks will only be awarded for the **FIRST** question in that section in the order the candidate submits the answers.

Candidates are allowed to use a calculator.

Answer questions only in the answer booklet provided.

UCCD3074 DEEP LEARNING FOR DATA SCIENCE**SECTION A (Answer all questions)**

Q1. Consider the following Gaussian function:

$$f(x) = -e^{-(x-2)^2}$$

- (a) Gradient descent with momentum is used to find the minima of $f(x)$ with the following settings: initial $x = 3$, initial $v = 0$, learning rate $\alpha = 0.5$ and momentum $\beta = 0.8$.

- (i) Explain how momentum helps to improve training efficiency. (2 marks)
- (ii) The optimizer finds the **global** minima for $f(x)$. What does that tell us about the function $f(x)$? (2 marks)
- (iii) The gradient is given by $\frac{df}{dx} = (2x - 4)(e^{-(x-2)^2})$. Perform gradient descent for the first 2 steps by completing the table below. Round to 4 decimal places in each step.

t	$x^{<t>}$	$f(x)^{<t>}$	$\frac{df^{<t>}}{dx}$	$v^{<t>}$
0	-	-	-	0
1	3			
2				

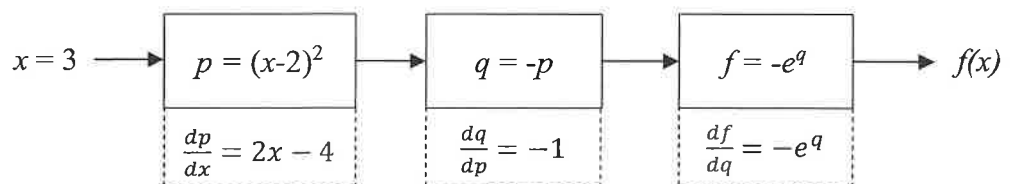
(9 marks)

- (iv) Consider gradient descent iteration $t = 10$, identify the weights of the current and previous gradients from $t = 10$ to 8 when computing $v^{<10>}$.

weight ^{<8>}	weight ^{<9>}	weight ^{<10>}

(3 marks)

- (b) Suppose that $f(x)$ is implemented with the following computational graph.



Given $x = 3$, perform forward and backpropagation to compute the gradient $\frac{df(x=3)}{dx}$. Show your steps. (9 marks)

[Total : 25 marks]

UCCD3074 DEEP LEARNING FOR DATA SCIENCE

- Q2. Given a credit dataset that classifies clients into good credit risk ($y = 0$) or bad ($y = 1$) credit risk. There are 1000 samples, with 800 good and 200 bad credits. A **standard** neural network (NN) is used to learn the task. The data set is a table with 1000 rows (each row is a sample) and 20 columns (listed in Table 2.1).

Transaction ID (int)	Employment status (int)	Present residence since (int)	Job (10 categories)
Name (String)	Installment rate (int)	Account status (5 categories)	Number of liable people to provide maintenance for (int)
Credit history (5 categories)	Marital status (binary)	Age (int)	Telephone (binary)
Purpose (5 categories)	Account type (5 categories)	Other installment plans (5 categories)	Nationality (5 categories)
Credit amount (int)	Housing (5 categories)	Number of existing credits at this bank (int)	Credit risk (binary)

Table 2.1: Column names for the table dataset

- (a) (i) Specify the necessary preprocessing steps to prepare the data for the model. Specify the shape of the batch data $\mathbf{X}$ with batch size 4. (6 marks)
- (ii) Design the standard neural network to handle the transaction. Your answer should define all the layers clearly, including their type, configuration and output shape as shown in the Table below.

Layer Type	Layer Configuration	Output Shape
...	...	...

(8 marks)

- (b) The standard neural network is trained for the task. The dataset is split into a training set (75%) and validation set (25%) using stratified sampling.
- (i) What is the appropriate cost function for the task? (2 marks)
- (ii) Your model achieves a validation accuracy of 80%. Discuss if the performance is acceptable. (3 marks)
- (iii) Suggest **TWO (2)** ways to improve the performance of the model. (4 marks)
- (iv) Although Conv1D can convolve over 1-D data type, justify why it should not be used for this task. (2 marks)
- [Total : 25 marks]

UCCD3074 DEEP LEARNING FOR DATA SCIENCE

- Q3. The model in Figure 3.1 is currently trained on RGB-D images (4 channels) of underwater scene. The network alerts the user if any of 6 targeted endangered fish species is present in the image. Different targeted species may appear together, or they may all be absent. Each batch data $\mathbf{X}$ contains 128 images with resolution 224×224 .

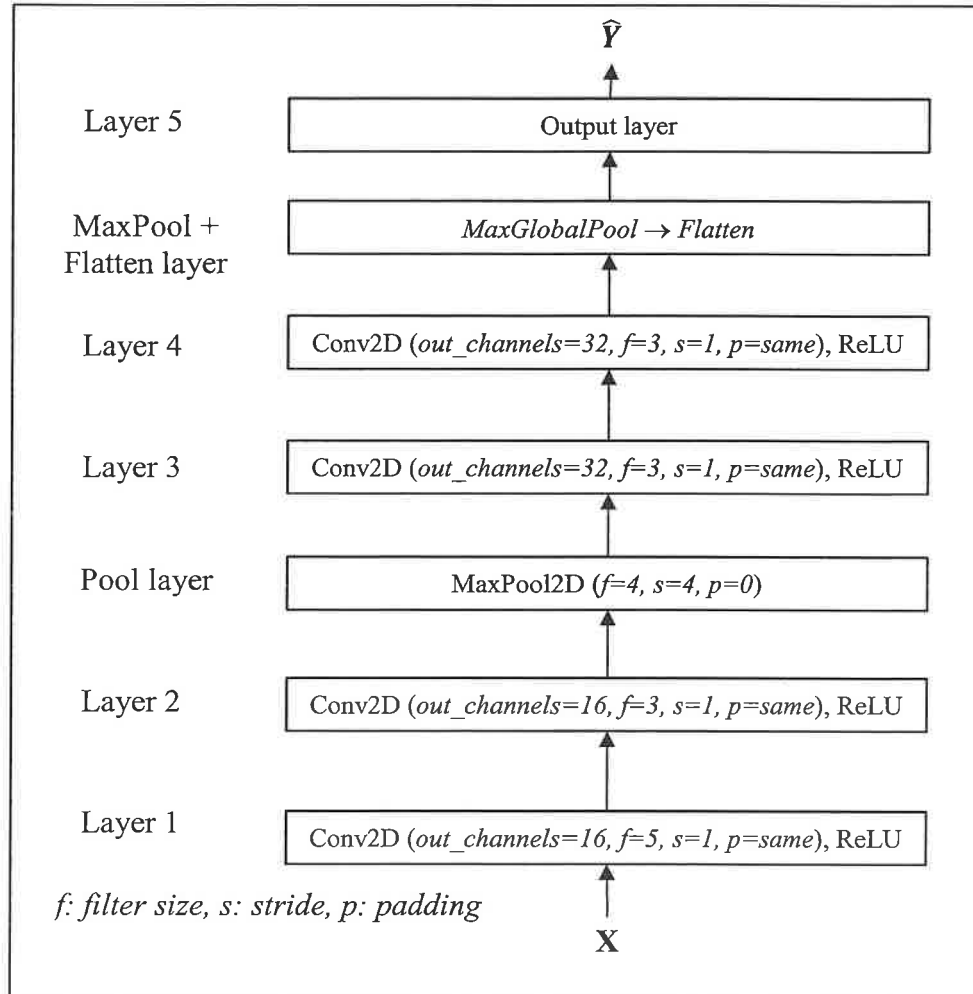


Figure 3.1: A CNN Network for fish segmentation

- (a) The following questions concerns the configuration of layer 1.
- What is the shape of the input matrix $\mathbf{X}$? (1 mark)
 - The same padding is used for all layers to preserve the input resolution. Compute the value of the padding p for layer 1. (2 marks)
 - What is the number of parameters in layer 1? (2 marks)
 - What is the number of computations in layer 1? (2 marks)
- (b) Specify the shape of the output tensor of all layers in Figure 3.1. (5 marks)

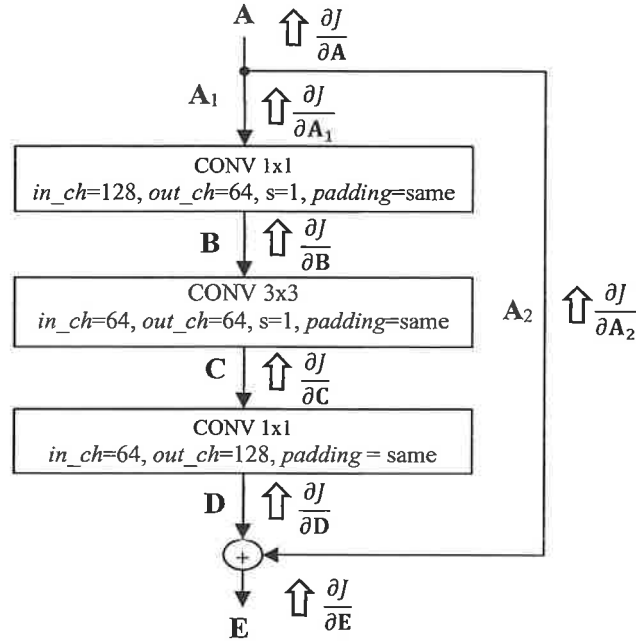
UCCD3074 DEEP LEARNING FOR DATA SCIENCE**Q3. (Continued)**

- (c) The following questions concerns the output layer and cost function.
- (i) Define Layer 5 to compute the output of the network. (2 marks)
 - (ii) What is the appropriate cost function to train the network? (2 marks)
- (d) The following questions concerns the input to the model.
- (i) Can the network handle image with different resolution? Justify your answer. (3 marks)
 - (ii) Can the model handle RGB input images (3 channels)? Justify your answer. (3 marks)
 - (iii) Suppose that the number of training samples is small. Would you increase or decrease the batch size? Justify your answer. (3 marks)
- [Total : 25 marks]

UCCD3074 DEEP LEARNING FOR DATA SCIENCE

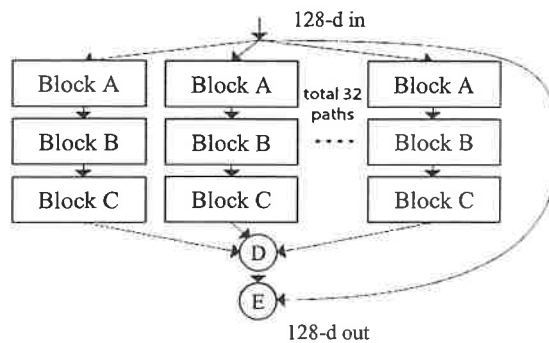
SECTION B (Choose ONE question only)

Q4. (a) The following figure shows a bottleneck residual.



- Suppose the shape of $A = (b, c, h, w) = (4, 128, 28, 28)$, what are the shapes of the tensors $\frac{\partial J}{\partial B}$, $\frac{\partial J}{\partial C}$, $\frac{\partial J}{\partial D}$ and $\frac{\partial J}{\partial E}$? (4 marks)
- Explain how the residual connection reduces the vanishing gradient effect at $\frac{\partial J}{\partial A}$. (3 marks)
- What are the roles of the two 1×1 CONV layers, respectively? (4 marks)
- Compare what the 3 CONV layers are learning with the residual connection compared to without the residual connection. (3 marks)

(b) Given a ResNeXt block with 32 paths with 4 channels per path as shown below:



- Specify how to implement blocks A, B and C. (6 marks)

UCCD3074 DEEP LEARNING FOR DATA SCIENCE**Q4. (b)(Continued)**

- (ii) Identify the operators D and E. (2 marks)
- (iii) How can we implement the 32 Block A with a single block? (3 marks)
- [Total : 25 marks]

- Q5. (a) (i) Compare self-supervised learning and traditional supervised learning. (4 marks)
- (ii) A pretrained transformer can be finetuned for different natural language processing tasks. Specify how to pretrain a transformer with the **masked language modeling** pretext task on a text corpus.

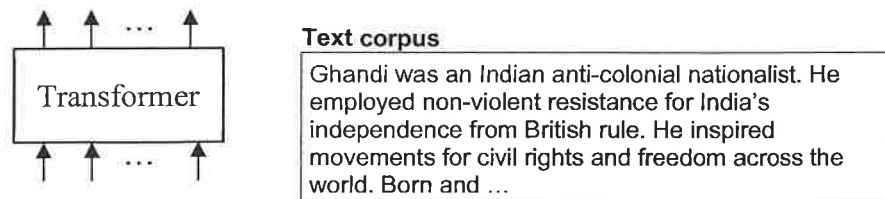
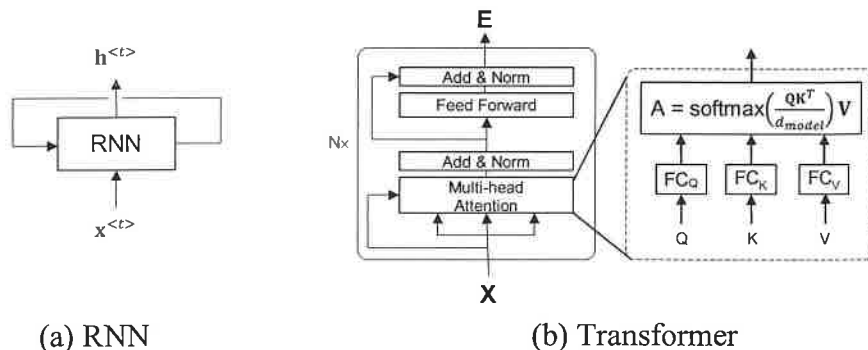


Figure 5.1: Transformer and Wikipedia corpus

Your answer should include:

- The masked language modeling method (how it works).
 - Example training samples (use the sentences in the text corpus in Figure 5.1)
 - The pretraining cost function
- (9 marks)

- (b) Figure 5.2 shows the RNN and Transformer modules.



(a) RNN

(b) Transformer

Figure 5.2: RNN vs Transformer

- (i) Explain why inference is slow in RNN but fast in Transformer. (4 marks)
- (ii) Discuss the impact of increasing the sequence length to the inference speed of FC_Q , FC_K and FC_V in the transformer? Justify your answer. (3 marks)

UCCD3074 DEEP LEARNING FOR DATA SCIENCE**Q5. (b)(Continued)**

- (iii) Discuss what row t of the matrix $\text{softmax}(\frac{QK^T}{d_{model}})$ represents. (3 marks)
- (iv) Given $batch_first = \text{True}$, $d_{model} = 64$, batch size $b = 32$ and sequence length = 50. What is the shape of $\mathbf{X}$ and $\mathbf{A}$ respectively? (2 marks)
- [Total : 25 marks]
-

Appendix: FormulasLogistic Regression:

$$z = w_1 x_1 + \dots + w_{N_x} x_{N_x} + b$$

$$= \mathbf{x}^T \mathbf{w} + b$$

$$z = \mathbf{Xw} + b$$

$$\hat{y} = \sigma(z) = \frac{1}{1+e^{-z}}$$

$$\hat{y} = \sigma(z) = \frac{1}{1+e^{-z}}$$

$$J(\mathbf{w}, b) = \frac{1}{M} \sum_{i=1}^M (-y^{(i)} \ln \hat{y}^{(i)} - (1 - y^{(i)}) \ln(1 - \hat{y}^{(i)}))$$

$$= \text{mean}((-y \cdot \ln \hat{y} - (1 - y) \cdot \ln(1 - \hat{y})))$$

$$\mathbf{w} := \mathbf{w} - \alpha \frac{\partial J(\mathbf{w}, b)}{\partial \mathbf{w}}$$

$$b := b - \alpha \frac{\partial J(\mathbf{w}, b)}{\partial b}$$

$$\frac{\partial J}{\partial \hat{y}} = \frac{1}{M} \left(\frac{1-y}{(1-\hat{y})} - \frac{y}{\hat{y}} \right)$$

$$\frac{\partial J}{\partial z} = \frac{\partial J}{\partial \hat{y}} \cdot (\hat{y} \cdot (1 - \hat{y}))$$

$$\frac{\partial J}{\partial \mathbf{w}} = \mathbf{X}^T \frac{\partial J}{\partial z}$$

$$\frac{\partial J}{\partial b} = \left(\mathbf{1}^T \frac{\partial J}{\partial z} \right)^T$$

Summation phase (1 sample)

Summation phase (batch data)

Cost function (1 sample)

Cost function (batch data)

Binary cost entropy

Update equation for gradient descent

Update equation for gradient descent

Backpropagation (cost layer)

Backpropagation (sigmoid activation layer)

Summation phase

Summation phase

Standard Neural Network:

$$\mathbf{Z}^{[l]} = \mathbf{A}^{[l-1]} \mathbf{W}^{[l]} + \mathbf{b}^{[l]}$$

$$\mathbf{A}^{[l]} = g(\mathbf{Z}^{[l]})$$

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^C e^{z_j}}$$

$$J(\mathbf{W}, \mathbf{b}) = \frac{1}{B} \sum_{i=1}^B -y^{(i)} \ln \hat{y}^{(i)} - (1 - y^{(i)}) \ln(1 - \hat{y}^{(i)})$$

$$= \text{mean}(-y \cdot \ln \hat{y} - (1 - y) \cdot \ln(1 - \hat{y}))$$

Summation phase

Activation phase

Softmax function

Binary cross entropy

$$J(\mathbf{W}, \mathbf{b}) = \frac{1}{C \times B} \sum_{c=1}^C \sum_{i=1}^B -y_c^{(i)} \ln \hat{y}_c^{(i)} - (1 - y_c^{(i)}) \ln(1 - \hat{y}_c^{(i)})$$

$$= \text{mean}(-\mathbf{Y} \cdot \ln \hat{\mathbf{Y}} - (1 - \mathbf{Y}) \cdot \ln(1 - \hat{\mathbf{Y}}))$$

Average binary cross entropy

$$J(\mathbf{W}, \mathbf{b}) = \frac{1}{B} \sum_{i=1}^B -\ln(p_c^{(i)})$$

Cross entropy function

$$\frac{\partial J}{\partial \hat{\mathbf{Y}}} = \frac{1}{C \times B} \left(\frac{1-\mathbf{Y}}{1-\hat{\mathbf{Y}}} - \frac{\mathbf{Y}}{\hat{\mathbf{Y}}} \right)$$

Backpropagation (cost layer for multi-label task)

$$\frac{\partial J}{\partial \mathbf{Z}^{[l]}} = \begin{cases} \frac{\partial J}{\partial \mathbf{A}^{[l]}} \cdot \mathbf{A}^{[l]} \cdot (1 - \mathbf{A}^{[l]}) & \text{for sigmoid activation} \\ \frac{\partial J}{\partial \mathbf{A}^{[l]}} \cdot (\mathbf{Z}^{[l]} > 0) & \text{for relu activation} \end{cases}$$

Backpropagation (activation layer)

$$\frac{\partial J}{\partial \mathbf{W}^{[l]}} = \mathbf{A}^{[l-1]T} \frac{\partial J}{\partial \mathbf{Z}^{[l]}}$$

Backpropagation (summation layer)

$$\frac{\partial J}{\partial \mathbf{b}^{[l]}} = \left(\mathbf{1}^T \frac{\partial J}{\partial \mathbf{Z}^{[l]}} \right)^T$$

Backpropagation (summation layer)

$$\frac{\partial J}{\partial \mathbf{A}^{[l-1]}} = \frac{\partial J}{\partial \mathbf{Z}^{[l]}} \mathbf{W}^{[l]T}$$

Backpropagation (summation layer)

Activation functions

$$g(z) = \frac{1}{1+e^{-z}}$$

sigmoid activation

$$\frac{dg}{dz} = g(z) \cdot (1 - g(z))$$

gradient for sigmoid

$$g(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

tanh activation

$$\frac{dg}{dz} = 1 - g(z)^2$$

gradient for tanh

$$g(z) = \max(0, z)$$

relu activation

$$\frac{dg}{dz} = \begin{cases} 1 & \text{if } z > 0 \\ \text{undefined} & \text{if } z = 0 \\ 0 & \text{if } z < 0 \end{cases}$$

gradient for relu

UCCD3074 DEEP LEARNING FOR DATA SCIENCE

$$g(z) = \max(0.01z, z)$$

leaky relu activation

$$\frac{dg}{dz} = \begin{cases} 1 & \text{if } z > 0 \\ \text{undefined} & \text{if } z = 0 \\ 0.01 & \text{if } z < 0 \end{cases}$$

gradient for leaky relu

Convolutional Neural Network:

$$A^{[l]}(i, j) = A_{loc}^{[l-1]}(i^*, j^*) \cdot W^{[l]} + b^{[l]}$$

Activation value at output location (i, j)

$$o = \left\lfloor \frac{i+2p-f}{s} + 1 \right\rfloor$$

size of the activation map

$$\# \text{params in a filter} = c_f \times h_f \times w_f + 1$$

Number of parameters in a filter

$$\# \text{params in a conv layer} = \# \text{filters} \times \# \text{params in a filter}$$

Number of parameters in a CONV layer

$$\begin{aligned} \# \text{computations} &= \# \text{output activations} \times \# \text{computations per activation} \\ &= \text{output volume size} \times \# \text{params in a filter} \end{aligned}$$

Number of computations in a CONV layer

$$\text{depth } d = \alpha^\phi, \text{ width } w = \beta^\phi, \text{ resolution } r = \gamma^\phi$$

Efficient Net formulation

$$\text{subject to: } \alpha, \beta^2, \gamma^2 \approx 2$$

$$\alpha \geq 1, \beta \geq 1, \gamma \geq 1$$

RNN:

$$H^{<t>} = g(H^{<t-1>} W_{hh} + X^{<t>} W_{hx} + b^T)$$

Simple RNN Cell

$$H^{<t>} = g([H^{<t-1>} \quad X^{<t>}] W_h + b^T)$$

Simple RNN Cell (Simplified form)

LSTM:

$$F^{<t>} = \sigma([H^{<t-1>} \quad X^{<t>}] W_f + b_f^T)$$

forget gate

$$I^{<t>} = \sigma([H^{<t-1>} \quad X^{<t>}] W_i + b_i^T)$$

input gate

$$O^{<t>} = \sigma([H^{<t-1>} \quad X^{<t>}] W_o + b_o^T)$$

output gate

$$\tilde{C}^{<t>} = \tanh([H^{<t-1>} \quad X^{<t>}] W_c + b_c^T)$$

new content (raw)

$$C^{<t>} = I^{<t>} \cdot \tilde{C}^{<t>} + F^{<t>} \cdot C^{<t-1>}$$

cell state

$$H^{<t>} = O^{<t>} \cdot \tanh(C^{<t>})$$

hidden state

Transformer:

$$A = \text{softmax}\left(\frac{QK^T}{\sqrt{d_{\text{model}}}}\right) V$$

self-attention module (without masking)

$$A_{\text{masked}} = \text{softmax}\left(\frac{QK^T}{\sqrt{d_{\text{model}}}} + M\right) V$$

self-attention module (with mask)

$$\bar{x}_{t,f} = \frac{x_{t,f} - \mu_{t,f}}{\sigma_{t,f}}$$

Layer normalization. Shape of X : (B, T, F)

$$\mu_{t,f} = \frac{1}{T \times F} \sum_t \sum_f x_{t,f}$$

Mean of Layer Norm. Shape of X : (B, T, F)

$$\sigma_{t,f} = \frac{1}{T \times F} \sqrt{\sum_t \sum_f (x_{t,f} - \mu_{t,f})^2}$$

Standard deviation of Layer Norm. Shape of X : (B, T, F) Preprocessing, regularization, batch normalization, initialization:

$$\bar{x}^{(i)} = \frac{x^{(i)} - \mu_M}{\sigma_M}$$

Standardization. Shape of X : (M, F)

$$\mu_M = \frac{1}{M} \sum_{i=1}^M x^{(i)}$$

Standardization's mean. Shape of X : (M, F)

$$\sigma_F = \frac{1}{M} \sum_{i=1}^M \sqrt{(x^{(i)} - \mu_M)^2}$$

Standardization's standard deviation. Shape of X : (M, F)

$$\bar{x}^{(i)} = \frac{x^{(i)} - \mu_B}{\sigma_B}$$

BatchNorm1D. Shape of X : (B, F)

$$\mu_B = \frac{1}{B} \sum_{b=1}^B x^{(i)}$$

BatchNorm1D's mean. Shape of X : (B, F)

$$\sigma_B = \frac{1}{B} \sum_{b=1}^B \sqrt{(x^{(i)} - \mu_B)^2}$$

BatchNorm1D's standard deviation. Shape of X : (B, F)

UCCD3074 DEEP LEARNING FOR DATA SCIENCE

$$\bar{z}_{h,w}^{(i)} = \frac{z_{h,w}^{(i)} - \mu_{B,H,W}}{\sqrt{v_{B,H,W} + \epsilon}}$$

$$\bar{z}_{h,w}^{(i)} = \gamma \cdot \bar{z}_{h,w}^{(i)} + \beta$$

$$\mu_{B,H,W} = \frac{1}{B \times H \times W} \sum_i \sum_h \sum_w z_{h,w}^{(i)}$$

$$v_{B,H,W} = \frac{1}{B \times H \times W} \sum_b \sum_h \sum_w (z_{h,w}^{(i)} - \mu_{B,H,W})^2$$

$$\bar{\mu}_{B,H,W} := (1 - m) \bar{\mu}_{B,H,W} + m \mu_{B,H,W}^{(t)}$$

$$\bar{v}_{B,H,W} := (1 - m) \bar{v}_{B,H,W} + m \left(\frac{B \times H \times W}{B \times H \times W - 1} v_{B,H,W}^{(t)} \right)$$

$$J_{reg}(W, b) = \frac{1}{m} \sum_{i=1}^m J(W, b | x^{(i)}, y^{(i)}) + \lambda \frac{1}{2m} \sum_{l=1}^L \|W^{[l]}\|_2^2$$

$$J_{reg}(W, b) = \frac{1}{m} \sum_{i=1}^m J(W, b | x^{(i)}, y^{(i)}) + \lambda \frac{1}{m} \sum_{l=1}^L |W^{[l]}|$$

$$var(W^{[l]}) = \frac{2}{(F^{[l-1]} + F^{[l]})}$$

$$var(W^{[l]}) = \frac{2}{F^{[l-1]}}$$

Optimizers

$$v_w^{<t>} := \beta v_w^{<t-1>} + (1 - \beta) \frac{\partial J}{\partial w}$$

$$v_w^{<t>} := \frac{1}{1 - \beta^t} v_w^{<t>}$$

$$w^{<t>} := w^{<t-1>} - \alpha v_w^{<t>}$$

$$weight^{<t-n>} = (1 - \beta) \beta^n$$

$$s_w^{<t>} := \beta s_w^{<t-1>} + (1 - \beta) \left(\frac{\partial J}{\partial w} \right)^2$$

$$w^{<t>} := w^{<t-1>} - \alpha \frac{\frac{\partial J}{\partial w}}{\sqrt{s_w^{<t>} + \epsilon}}$$

$$m^{<t>} := \beta_1 m^{<t-1>} + (1 - \beta_1) \frac{\partial J}{\partial w}$$

$$\tilde{m}_w^{<t>} = \frac{m_w^{<t>}}{(1 - \beta_1^t)}$$

$$v_w^{<t>} := \beta_2 v_w^{<t-1>} + (1 - \beta_2) \left(\frac{\partial J}{\partial w} \right)^2$$

$$\tilde{v}_w^{<t>} = \frac{v_w^{<t>}}{(1 - \beta_2^t)}$$

$$w^{<t>} = w^{<t-1>} - \frac{\alpha}{\sqrt{v_w^{<t>} + \epsilon}} \tilde{m}_w^{<t>}$$

Learning rate decay

$$\alpha^{<t>} = \frac{1}{1 + \tau \times t} \alpha^{<0>}$$

$$\alpha^{<t>} = \gamma^{t/T} \times \alpha^{<0>}$$

$$\alpha^{<t>} = e^{-k \times t} \times \alpha^{<0>}$$

BatchNorm2D (1st part). Shape of Z : (B, C, H, W)

BatchNorm2D (2nd part). Shape of Z : (B, C, H, W)

BatchNorm2D's mean during training. Shape of Z : (B, C, H, W)

BatchNorm2D's variance during training. Shape of Z : (B, C, H, W)

BatchNorm2D's running mean during training.

BatchNorm2D's running variance during training.

L2 regularization for an L -layered network, m samples

L1 regularization for an L -layered network, m samples

Xavier initialization

Kaiming initialization

Momentum term

Momentum: Bias correction

Momentum: Update equation

Momentum: Weight for n -th previous iterations at time step $t-n$

RMSProp: Moving average of squared gradient

RMSProp: update equation

Adam: momentum term

Adam: rmsprop term

Adam: Update equation

Time-based decay

Step decay schedule

Exponential decay

